

Welcome to CSC331 Fall 2025 Data Structures

Professor Kevin Byron (kbyron@bmcc.cuny.edu)

Department of CIS, Room F-1030-I

Office hours: Mon 2pm-5pm (in-person and Zoom)

Office phone: 212-220-1490

Borough of Manhattan Community College/CUNY

CSC331 Fall 2025

Data Structures

Week 2:

Pointers

Array-based lists

Linked lists

Program assignment 2

Professor Kevin Byron

Department of CIS

Borough of Manhattan Community College/CUNY

Reminders

- No class Mon 9-1-2025, Mon 9-22-2025 thru Wed 9-24-2025
- Program 1 due date
 - 1159pm Fri 9-19-2025
 - late submission receives no credit
- Exam 1
 - Section 501 Tue 9-30-2025 – Zoom 630pm-730pm
 - Section 172 Wed 10-8-2025 – In person 630pm-730pm
 - Closed book, closed notes, no devices
 - Section 500 Wed 10-8-2025 – In person 1010am-1110am
 - Closed book, closed notes, no devices
- Program 2 due date
 - 1159pm Fri 10-10-2025
 - late submission receives no credit
- Posted on Brightspace
 - Week 2 lecture slides, C++ program with pointers as parameters (dm377.cpp), buildListForward function implementation (chap5a.cpp), program 2 specifications, book_f25.h header file

Pointers and array-based lists

The data types in C++ are classified into three categories: simple, structured, and pointers. Until now, you have worked with only the first two data types. This chapter discusses the third data type: the pointer data type. You first learn how to declare pointer variables (or pointers, for short) and manipulate the data to which they point. Later, you use these concepts when you study dynamic arrays and linked lists. Linked lists are discussed in Chapter 5.

Before discussing how pointers work, let us make the following observations. The following statements that declare `p` to be a pointer variable of type `int` are equivalent:

```
int *p;  
int*  p;  
int * p;
```

In this statement, only `p` is a pointer variable, not `q`. Here `q` is an `int` variable. To avoid confusion, we prefer to attach the character `*` to the variable name. So the preceding statement is written as follows:

```
int *p, q;
```

Pointers and array-based lists

In C++, the ampersand, `&`, called the **address of operator**, is a unary operator that returns the address of its operand. For example, given the statements

```
int x;  
int *p;
```

the statement

```
p = &x;
```

assigns the address of `x` to `p`. That is, `x` and the value of `p` refer to the same memory location.

```
int x = 25;  
int *p;  
p = &x;    //store the address of x in p
```

the statement

```
cout << *p << endl;
```

prints the value stored in the memory space to which `p` points, which is the value of `x`. Also, the statement

```
*p = 55;
```

stores 55 in the memory location to which `p` points—that is, 55 is stored in `x`.

Pointers and array-based lists

```
int main() //Line 3
{ //Line 4
    int *p; //Line 5
    int num1 = 5; //Line 6
    int num2 = 8; //Line 7

    p = &num1; //store the address of num1 into p; Line 8

    cout << "Line 9: &num1 = " << &num1
          << ", p = " << p << endl; //Line 9
    cout << "Line 10: num1 = " << num1
          << ", *p = " << *p << endl; //Line 10

    *p = 10; //Line 11
    cout << "Line 12: num1 = " << num1
          << ", *p = " << *p << endl << endl; //Line 12

    p = &num2; //store the address of num2 into p; Line 13

    cout << "Line 14: &num2 = " << &num2
          << ", p = " << p << endl; //Line 14
    cout << "Line 15: num2 = " << num2
          << ", *p = " << *p << endl; //Line 15

    *p = 2 * (*p); //Line 16
    cout << "Line 17: num2 = " << num2
          << ", *p = " << *p << endl; //Line 17

    return 0; //Line 18
} //Line 19
```

Pointers and array-based lists

Sample Run:

Line 9: &num1 = 0012FF54, p = 0012FF54

Line 10: num1 = 5, *p = 5

Line 12: num1 = 10, *p = 10

Line 14: &num2 = 0012FF48, p = 0012FF48

Line 15: num2 = 8, *p = 8

Line 17: num2 = 16, *p = 16

Pointers and array-based lists

Pointers and Classes

Consider the following statements:

```
string *str;  
str = new string;  
*str = "Sunny Day";
```

The first statement declares `str` to be a pointer variable of type `string`. The second statement allocates memory of type `string` and stores the address of the allocated memory in `str`. The third statement stores the string "Sunny Day" in the memory to which `str` points. Now suppose that you want to use the string function `length` to find the length of the string "Sunny Day". The statement `(*str).length()` returns the length of the string.

Pointers and array-based lists

Operator **new**

The operator **new** has two forms: one to allocate a single variable, and another to allocate an array of variables. The syntax to use the operator **new** is as follows:

```
new dataType;           //to allocate a single variable  
new dataType[intExp];   //to allocate an array of variables
```

where **intExp** is any expression evaluating to a positive integer.

Pointers and array-based lists

The operator `new` allocates memory (a variable) of the designated type and returns a pointer to it—that is, the address of this allocated memory. Moreover, the allocated memory is uninitialized.

Consider the following declaration:

```
int *p;  
char *q;  
int x;
```

The statement

```
p = &x;
```

stores the address of `x` in `p`. However, no new memory is allocated. On the other hand, consider the following statement:

```
p = new int;
```

This statement creates a variable during program execution somewhere in memory, and stores the address of the allocated memory in `p`. The allocated memory is accessed via pointer dereferencing—namely, `*p`. Similarly, the statement

```
q = new char[16];
```

creates an array of 16 components of type `char` and stores the base address of the array in `q`.

Pointers and array-based lists

```
int main() //Line 3
{ //Line 4
    int *p; //Line 5
    int *q; //Line 6

    p = new int; //Line 7
    *p = 34; //Line 8
    cout << "Line 9: p = " << p //Line 9
        << ", *p = " << *p << endl;

    q = p; //Line 10
    cout << "Line 11: q = " << q //Line 11
        << ", *q = " << *q << endl;

    *q = 45; //Line 12
    cout << "Line 13: p = " << p //Line 13
        << ", *p = " << *p << endl;
    cout << "Line 14: q = " << q //Line 14
        << ", *q = " << *q << endl;

    p = new int; //Line 15
    *p = 18; //Line 16
    cout << "Line 17: p = " << p //Line 17
        << ", *p = " << *p << endl;
    cout << "Line 18: q = " << q //Line 18
        << ", *q = " << *q << endl;
```

Pointers and array-based lists

```
delete q; //Line 19
q = NULL; //Line 20
q = new int; //Line 21
*q = 62; //Line 22
cout << "Line 23: p = " << p
      << ", *p = " << *p << endl; //Line 23
cout << "Line 24: q = " << q
      << ", *q = " << *q << endl; //Line 24

return 0; //Line 25
} //Line 26
```

Sample Run:

```
Line 9: p = 00355620, *p = 34
Line 11: q = 00355620, *q = 34
Line 13: p = 00355620, *p = 45
Line 14: q = 00355620, *q = 45
Line 17: p = 003556C8, *p = 18
Line 18: q = 00355620, *q = 45
Line 23: p = 003556C8, *p = 18
Line 24: q = 00355620, *q = 62
```

Pointers and array-based lists

The arithmetic operations that are allowed differ from the arithmetic operations on numbers. First, let us use the following statements to explain the increment and decrement operations on pointer variables:

```
int *p;  
double *q;  
char *chPtr;
```

Suppose that the size of the memory allocated for an `int` variable is 4 bytes, a `double` variable is 8 bytes, and a `char` variable is 1 byte.

The statement

```
p++; or p = p + 1;
```

increments the value of `p` by 4 bytes because `p` is a pointer of type `int`. Similarly, the statements

```
q++;  
chPtr++;
```

increment the value of `q` by 8 bytes and the value of `chPtr` by 1 byte, respectively.

Pointers and array-based lists

An array created during the execution of a program is called a **dynamic array**. To create a dynamic array, we use the second form of the **new** operator.

The statement

```
int *p;
```

declares **p** to be a pointer variable of type **int**. The statement

```
p = new int[10];
```

allocates 10 contiguous memory locations, each of type **int**, and stores the address of the first memory location into **p**. In other words, the operator **new** creates an array of 10 components of type **int**, it returns the base address of the array, and the assignment operator stores the base address of the array into **p**. Thus, the statement

```
*p = 25;
```

stores 25 into the first memory location, and the statements

```
p++;          //p points to the next array component  
*p = 35;
```

Pointers and array-based lists

store 35 into the second memory location. Thus, by using the increment and decrement operations, you can access the components of the array. Of course, after performing a few increment operations, it is possible to lose track of the first array component. C++ allows us to use array notation to access these memory locations. For example, the statements

```
p[0] = 25;  
p[1] = 35;
```

store 25 and 35 into the first and second array components, respectively. That is, `p[0]` refers to the first array component, `p[1]` refers to the second array component, and so on.

Pointers and array-based lists

Functions and Pointers

A pointer variable can be passed as a parameter to a function either by value or by reference. To declare a pointer as a value parameter in a function heading, you use the same mechanism as you use to declare a variable. To make a formal parameter be a reference parameter, you use `&` when you declare the formal parameter in the function heading. Therefore, to declare a formal parameter as a reference parameter, you must use `&`. Between the data type name and the identifier name, you must include `*` to make the identifier a pointer and `&` to make it a reference parameter. The obvious question is: In what order should `&` and `*` appear between the data type name and the identifier to declare a pointer as a reference parameter? In C++, to make a pointer a reference parameter in a function heading, `*` appears before the `&` between the data type name and the identifier. The following example illustrates this concept:

```
void example(int* &p, double *q)
{
    .
    .
    .
}
```


Pointers and array-based lists

Show the output from the following code ...

```
void example(int* &p, double* q){
    cout << "p = " << *p << endl;
    cout << "q = " << *q << endl;
    p = new int;
    q = new double;
    *p = 74;
    *q = 6.28;
}

int main(){
    int *a;
    double *b;
    a = new int;
    b = new double;
    *a = 37;
    *b = 3.14;
    example(a,b);
    cout << "a = " << *a << endl;
    cout << "b = " << *b << endl;
}
```

Pointers and array-based lists

Output from code on previous slide ...

```
p = 37  
q = 3.14  
a = 74  
b = 3.14
```

Pointers and array-based lists

Array-Based Lists

Everyone is familiar with the term *list*. You might have a list consisting of employee data, student data, sales data, or a list of rental properties. One thing common to all lists is that all the elements of a list are of the same type. More formally, we can define a list as follows:

List: A collection of elements of the same type.

The **length** of a list is the number of elements in the list.

Pointers and array-based lists

Following are some of the operations performed on a list:

1. Create the list. The list is initialized to an empty state.
2. Determine whether the list is empty.
3. Determine whether the list is full.
4. Find the size of the list.
5. Destroy, or clear, the list.
6. Determine whether an item is the same as a given list element.
7. Insert an item in the list at the specified location.
8. Remove an item from the list at the specified location.
9. Replace an item at the specified location with another item.
10. Retrieve an item from the list from the specified location.
11. Search the list for a given item.

Linked lists

A linked list is a collection of components, called **nodes**. Every node (except the last node) contains the address of the next node. Thus, every node in a linked list has two components: one to store the relevant information (that is, data) and one to store the address, called the **link**, of the next node in the list. The address of the first node in the list is stored in a separate location, called the **head** or **first**. Figure 5-1 is a pictorial representation of a node.



Linked lists

Because each node of a linked list has two components, we need to declare each node as a `class` or `struct`. The data type of each node depends on the specific application—that is, what kind of data is being processed. However, the link component of each node is a pointer. The data type of this pointer variable is the node type itself. For the previous linked list, the definition of the node is as follows. (Suppose that the data type is `int`.)

```
struct nodeType
{
    int info;
    nodeType *link;
};
```

The variable declaration is as follows:

```
nodeType *head;
```

Linked lists

Suppose that `p` points to the node with `info` 65, and a new node with `info` 50 is to be created and inserted after `p`. Consider the following statements:

```
newNode = new nodeType; //create newNode
newNode->info = 50;      //store 50 in the new node
newNode->link = p->link;
p->link = newNode;
```

Linked lists

BUILDING A LINKED LIST FORWARD

Suppose that the nodes are in the usual `info-link` form and `info` is of type `int`. Let us assume that we process the following data:

2 15 8 24 34

We need three pointers to build the list: one to point to the first node in the list, which cannot be moved, one to point to the last node in the list, and one to create the new node. Consider the following variable declaration:

```
nodeType *first, *last, *newNode;  
int num;
```

Suppose that `first` points to the first node in the list. Initially, the list is empty, so both `first` and `last` are `NULL`. Thus, we must have the statements

```
first = NULL;  
last = NULL;
```

to initialize `first` and `last` to `NULL`.

Linked lists

```
1  cin >> num;           //read and store a number in num
2  newNode = new nodeType; //allocate memory of type nodeType
                           //and store the address of the
                           //allocated memory in newNode
3  newNode->info = num;    //copy the value of num into the
                           //info field of newNode
4  newNode->link = NULL;   //initialize the link field of
                           //newNode to NULL
5  if (first == NULL)     //if first is NULL, the list is empty;
                           //make first and last point to newNode
    {
5a    first = newNode;
5b    last = newNode;
    }
6  else                   //list is not empty
    {
6a    last->link = newNode; //insert newNode at the end of the list
6b    last = newNode;      //set last so that it points to the
                           //actual last node in the list
    }
```

Linked lists

For example, suppose that head points to a linked list of numbers. The following code outputs the data stored in each node:

```
current = head;

while (current != NULL)
{
    cout << current->info << " ";
    current = current->link;
}
```

Linked lists

```
nodeType* buildListForward()
{
    nodeType *first, *newNode, *last;
    int num;

    cout << "Enter a list of integers ending with -999."
         << endl;
    cin >> num;
    first = NULL;

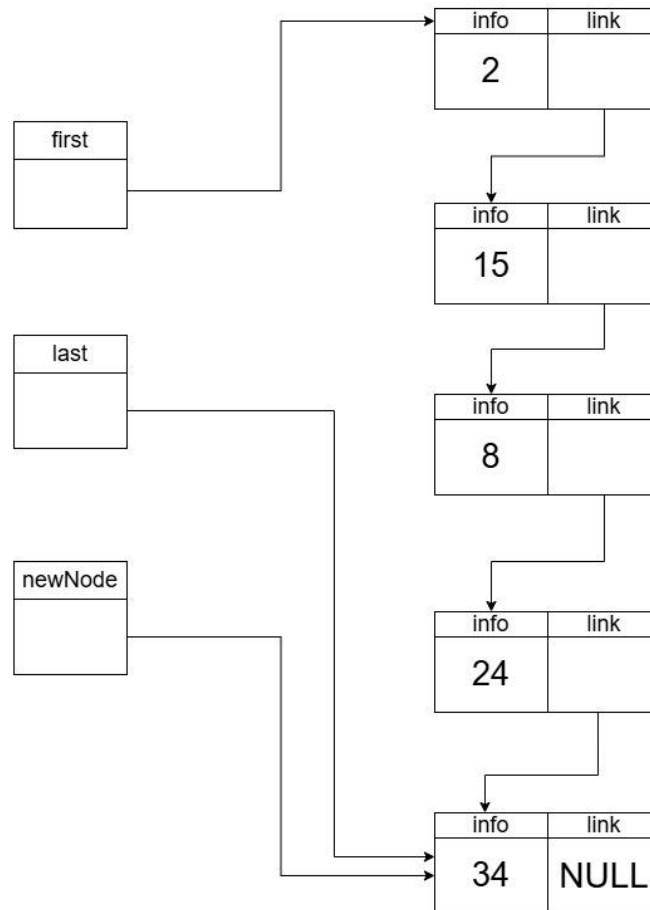
    while (num != -999)
    {
        newNode = new nodeType;
        newNode->info = num;
        newNode->link = NULL;

        if (first == NULL)
        {
            first = newNode;
            last = newNode;
        }
        else
        {
            last->link = newNode;
            last = newNode;
        }
        cin >> num;
    } //end while

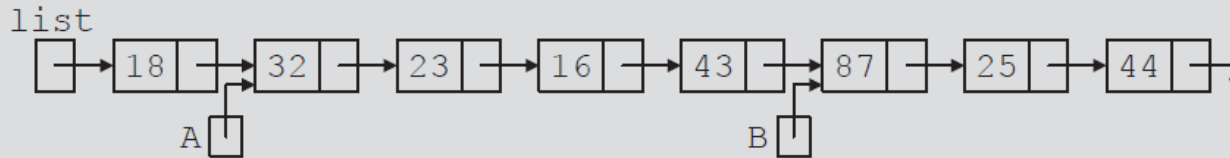
    return first;
} //end buildListForward
```

Linked lists

Linked list produced by buildListForward function ...



Linked lists



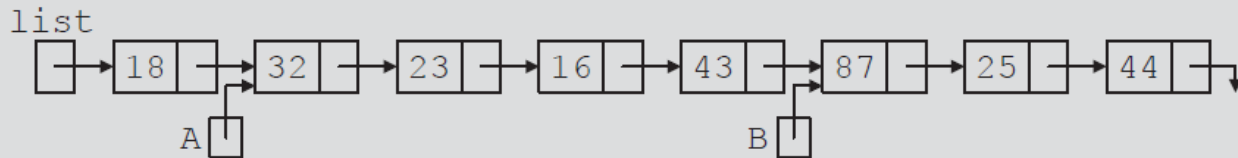
What is the output of each of the following C++ statements?

- a. `cout << list->info;`
- b. `cout << A->info;`
- c. `cout << B->link->info;`
- d. `cout << list->link->link->info`

What is the value of each of the following relational expressions?

- a. `list->info >= 18`
- b. `list->link == A`
- c. `A->link->info == 16`
- d. `B->link == NULL`
- e. `list->info == 18`

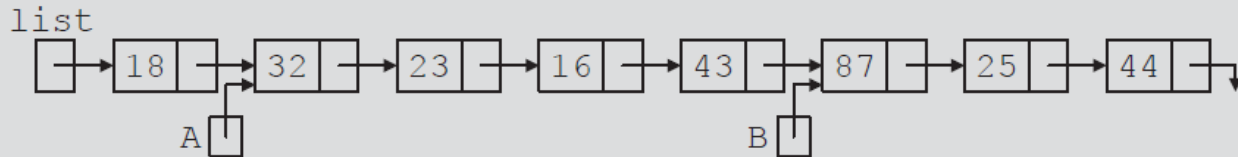
Linked lists



Mark each of the following statements as valid or invalid. If a statement is invalid, explain why.

- a. `A = B;`
- b. `list->link = A->link;`
- c. `list->link->info = 45;`
- d. `*list = B;`
- e. `*A = *B;`
- f. `B = A->link->info;`
- g. `A->info = B->info;`
- h. `list = B->link->link;`
- i. `B = B->link->link->link;`

Linked lists



Write C++ statements to do the following:

- Make **A** point to the node containing **info 23**.
- Make **list** point to the node containing **16**.
- Make **B** point to the last node in the list.
- Make **list** point to an empty list.
- Set the value of the node containing **25** to **35**.
- Create and insert the node with **info 10** after the node pointed to by **A**.
- Delete the node with **info 23**. Also, deallocate the memory occupied by this node.

Assignment

- Read Malik chapter 3 – pointers and dynamic memory allocation
- Try Chapter 3 Exercises 1-10 for practice – do not submit
- Read Malik chapter 5 - linked lists